

IFSP - INSTITUTO FEDERAL DE SÃO PAULO

Daniel Shoji Cornejo Fujita

Maria Eduarda Oliveira

Maysa Vieira da Silva

Rodrigo Makoto Gonçalves Kanezuka

Thainá Bezerra Lima

PROJETO DO JOGO
TURING'S CODE

SÃO PAULO

2026

IFSP INSTITUTO FEDERAL DE SÃO PAULO

Projeto de Jogo do 1º ano do Ensino Médio Integrado em Desenvolvimento de Sistemas da Turma 131, do IFSP Instituto Federal de São Paulo, realizado sob orientação da professora Miyuki.

SÃO PAULO

2026

Dedicamos este trabalho às famílias e aos moradores que confiam diariamente no SUS e que enfrentam desafios em busca de um atendimento ideal. Também dedicamos a todos aqueles que lutam por um sistema mais acessível e humano, e que está pesquisa contribua para a compreensão da sua relevância e inspire iniciativas que fortaleçam um sistema de saúde melhor.

EPÍGRAFE

"Às vezes, são as pessoas de quem ninguém espera nada que fazem as coisas que ninguém consegue imaginar."

— Alan Turing

"Tudo o que é digital hoje já foi engrenagem, relé e circuito ontem."

— Turing's Code

"A tecnologia do presente não nasceu em telas reluzentes, mas na lógica, no suor e na coragem daqueles que desbravaram os primeiros códigos da história."

"A computação não é sobre máquinas, assim como a astronomia não é sobre telescópios; é sobre a arte de pensar com ordem e lógica."

RESUMO

O presente documento apresenta a especificação técnica, conceitual e artística do Turing 's Code, um jogo digital educativo do gênero *Puzzle Adventure* desenvolvido por estudantes do 1.º ano do Ensino Técnico Integrado em Desenvolvimento de Sistemas do IFSP (Campus São Paulo). O projeto tem como objetivo principal conectar a aprendizagem técnica à cultura de jogos eletrônicos (*Game-Based Learning*), direcionando-se a jovens do ensino médio e jovens adultos.

Utilizando o motor gráfico GameMaker (via linguagem GML) e uma direção de arte inspirada em *pixel-art* retrô, a narrativa e as mecânicas do jogo resgatam o contexto histórico de Bletchley Park e os primórdios da computação durante a Segunda Guerra Mundial. O enredo e a jogabilidade estruturam-se na cooperação interdisciplinar entre a lógica de software (representada pelo protagonista Alan Torres) e a engenharia de hardware e analógica (representada pelo co-protagonista Théo Torres), integrando conteúdos de Arquitetura de Computadores, Lógica de Programação e Matemática. Como documento orientador, este GDD detalha os fundamentos teóricos, o design de mecânicas e fases, a arquitetura de sistema, além do planejamento voltado à prototipagem, testes de usabilidade e depuração pedagógica.

SUMÁRIO

INTRODUÇÃO 5

OBJETIVOS 6

CAPÍTULO 1 - FUNDAMENTAÇÃO TEÓRICA 7

1.1 Jogos educativos e Gamificação 7

1.2 Fundamentos de Game Design 7

1.3 Tecnologias e Ferramentas 9

CAPÍTULO 2 - CONCEPÇÃO DO JOGO 10

2.1 Visão Geral do Jogo

2.2 História e Narrativa 12

2.2.1 Personagens 12

2.2.2 StoryBoard 13

2.3 Fases do jogo : Mecânicas e Regras de Jogabilidade 14

2.4 Level Design 14

2.5 Direção de Arte e Som 14

CAPÍTULO 3 - ARQUITETURA E DESENVOLVIMENTO TÉCNICO 14

3.1 Arquitetura do Sistema 14

3.2 Interação Interdisciplinar 15

15

3.3 Implementação do protótipo

CAPÍTULO 4 - TESTES E RESULTADOS 15

4.1 Metodologia dos testes 15

4.2 Análise dos resultados 15

4.3 Ajustes e melhorias Realizadas

CAPÍTULO 5 - CONSIDERAÇÕES FINAIS 16

5.1 Conclusões 16

5.2 Dificuldades Encontradas

5.3 Possíveis alterações

PÓS-TEXTUAIS 18

18

18

REFERÊNCIAS BIBLIOGRÁFICAS 18

19

19

19

INTRODUÇÃO

Nos últimos anos, a indústria de jogos eletrônicos se estabeleceu como a principal vertente do entretenimento global, destacando-se pelo alto faturamento, avanços tecnológicos e transformações no modo como as pessoas interagem. Atualmente, essa mudança impacta diretamente a educação, com a nova geração de estudantes cada vez mais conectada à tecnologia, os jogos têm disputado a atenção com o ambiente escolar.

Portanto, torna-se necessário transformar esse cenário. Com o surgimento contínuo de novas tecnologias, a tendência é que essa competição se intensifique. Diante desse contexto, a criação de jogos educativos surge como uma solução didática e promissora.

Nesse contexto, os estudantes do 1.º ano do Ensino Técnico Integrado em Desenvolvimento de Sistemas realizam um projeto de jogo que conecta a educação ao universo gamer. A iniciativa permite que os estudantes deixem de ser apenas usuários e se tornem agentes ativos no processo de desenvolvimento, exigindo a integração das disciplinas do curso técnico — tais como game design, arquitetura de computadores, arte visual, ferramentas de desenvolvimento, desenvolvimento web e linguagens de programação.

Nessa fusão entre tecnologia e linguagem educacional, este Game Design Document (GDD) apresenta a especificação técnica, conceitual e artística para o desenvolvimento do jogo Lógica de Turing. O título pertence ao gênero educativo, projetado para ser disponibilizado no GitHub, desenvolvido no motor gráfico GameMaker e direcionado ao público-alvo de jovens do ensino médio e jovens adultos que buscam desafios instigantes.

A proposta central de Lógica de Turing é explorar o raciocínio lógico dos estudantes, propondo desafios baseados nos conteúdos de Arquitetura de Computadores, Lógica de Programação e Matemática, aliados a uma narrativa vinculada à história de Alan Turing para promover o conhecimento histórico sobre o "pai da computação". Por meio de uma dinâmica de jogabilidade, level design progressivo e uma identidade estética histórica, o projeto visa entregar uma experiência divertida e educativa, na qual cada mecânica projetada atenda aos objetivos teóricos estabelecidos nesta documentação.

Como documento orientador do projeto, este GDD cumpre o propósito de formalizar os requisitos de design, sistematizar as regras e dinâmicas do sistema, bem como orientar a produção dos ativos visuais, sonoros e de programação necessários para a execução do produto final.

OBJETIVOS

1. OBJETIVO GERAL

Desenvolver o jogo educativo Lógica de Turing, direcionado a jovens do Ensino Médio e jovens adultos, relacionando conceitos de Arquitetura de Computadores, Lógica de Programação e Matemática a uma narrativa que conecta a atualidade à história de Alan Turing durante a Segunda Guerra Mundial.

2. OBJETIVOS ESPECÍFICOS

1. Pilar de Game Design & Mecânicas:

O design de jogo de *Lógica de Turing* fundamenta-se na união simbiótica entre a resolução mecânica de desafios lógicos e a narrativa histórica, garantindo que o desafio intelectual estimule o raciocínio sem gerar frustração excessiva (*fun factor*). As mecânicas são construídas para refletir a colaboração entre os dois protagonistas — a mente lógica de Alan e a engenharia analógica de Theo:

- **Curva de Dificuldade e Fluxo (*Flow*):** Segundo a teoria do fluxo de Mihaly Csikszentmihalyi, o engajamento ideal ocorre quando há equilíbrio entre o desafio e a habilidade do jogador. No jogo, essa premissa guia a progressão modular dos quebra-cabeças, que introduzem conceitos lógicos e de arquitetura de computadores de forma gradual (indo do concreto analógico ao abstrato digital).

- **Mecânicas Híbridas (Hardware vs. Software):** As mecânicas centrais exigem que o jogador alterne e combine as especialidades dos personagens. Enquanto Alan manipula linhas de código, variáveis e fluxos lógicos, Theo interage diretamente com a tactilidade de engrenagens, rotores mecânicos e cabos de conexão (inspirados nas máquinas Enigma e Bombe), provando que a tecnologia moderna e o passado mecânico se complementam.

- **Mecânicas e *Feedback* Imediato (*Debugging* Lúdico):** O projeto prioriza respostas visuais e sonoras instantâneas. Sempre que o jogador manipula uma engrenagem, um cabo no *Hut* ou uma linha de lógica, o sistema reage

de maneira clara. Isso transforma o erro em uma ferramenta pedagógica de tentativa, erro e depuração (*debugging*) em um ambiente seguro e imersivo.

- **Narrativa Ambiental e Lúdica:** A atmosfera de sigilo e inteligência de Bletchley Park é integrada diretamente ao design de missões e à interface (UI/UX). Elementos visuais nostálgicos e monitores, contextualizando o aprendizado técnico, unindo a história da computação à experiência interativa.

2. **Pilar Artístico e Narrativo:**

Pilar Artístico: Sprints nostálgicas

- **Pixel Art :** A estética visual é inspirada em imagens reais da época em que Turing vivia, utilizado imagens de referência, e paletas de cores que dão essa vibe de coisas mais antigas.

- **Tactilidade Visual (Interface Tátil):** As engrenagens da Enigma, os cabos do Hut 6 e as chaves da máquina Bombe possuem peso e detalhamento tátil em pixel art, fazendo o jogador sentir a resistência física dos aparelhos analógicos de época.

- **Trilha Sonora e Efeitos Dissonantes:**

Pilar Narrativo: A Ponte entre o Passado do hardware e o Futuro dos Codes:

- **Complementaridade de Gerações (Hardware vs. Software):** A história mostra que nem a tecnologia moderna nem o conhecimento antigo funcionam isoladamente. O avanço só ocorre quando a percepção física do Theo

(hardware/geometria) se une ao raciocínio algorítmico do Alan (software/lógica).

- **Homenagem à Origem da Computação:** A narrativa desmistifica a tecnologia atual ao mostrar que os conceitos de lógica não nasceram nas telas de computador, mas no suor de Bletchley Park e nas invenções de Alan Turing durante a Segunda Guerra Mundial.

- **Evolução e Respeito Mútuo:** A jornada transforma os dois personagens: Alan aprende a desacelerar e respeitar a complexidade do mundo físico, enquanto Theo percebe que o futuro da mecânica está na velocidade dos códigos.

3. **Pilar Técnico e Pedagógico:**

- **Motor de Jogo e Otimização Tecnológica:** O desenvolvimento é estruturado no GameMaker, motor escolhido por sua alta performance em renderização 2D em pixel-art e flexibilidade na programação via *GameMaker Language (GML)*. Isso garante um controle rigoroso sobre colisões, manipulação de sprites e a criação de scripts modulares para os simuladores de lógica e desafios.

- **Fundamentos de Arquitetura, Lógica e Matemática:** Os desafios do jogo são construídos com base em conceitos reais do curso técnico em Desenvolvimento de Sistemas, abordando estruturas condicionais, e manipulação de variáveis, princípios básicos de redes e arquitetura de computadores e matemática básica.

- **Aprendizagem Baseada em Jogos (*Game-Based Learning - GBL*):** O projeto fundamenta-se na premissa de que o aprendizado técnico é mais efetivo quando integrado diretamente à mecânica interativa (*core loop*), permitindo que o aluno compreenda a lógica de programação de forma aplicada e contextualizada historicamente.

- **Ciclo de Tentativa, Erro e *Debugging* Pedagógico:** O design pedagógico promove o "erro seguro", onde falhar em uma sequência lógica ou conexão de cabos não gera penalidades severas, mas serve como *feedback* visual imediato para que o estudante analise o problema, refaça o raciocínio e pratique o conceito de depuração (*debugging*) de forma lúdica.

CAPÍTULO 1 - FUNDAMENTAÇÃO TEÓRICA

1.1 Jogos educativos

Os jogos educativos, frequentemente associados ao conceito de *Serious Games* (Jogos Sérios), são sistemas interativos projetados com a finalidade primária de promover o aprendizado e a aquisição de habilidades, sem abdicar do entretenimento e do engajamento inerentes aos jogos eletrônicos (Prensky, 2001).

De acordo com Gee (2003), os jogos digitais possuem uma arquitetura natural para o aprendizado, pois colocam o jogador como agente ativo em um ambiente simulado. Diferente dos métodos passivos tradicionais, o aprendizado baseado em jogos (*Game-Based Learning - GBL*) permite a experimentação por tentativa e erro em um ambiente livre de riscos reais, encorajando a tomada de decisão e a resolução de problemas.

Neste projeto, a abordagem adotada busca integrar o conteúdo pedagógico de [Inserir Matéria/Assunto, ex: *Física Clássica*] diretamente às mecânicas core do jogo. Dessa forma, o progresso do jogador depende da compreensão e aplicação prática dos conceitos ensinados, garantindo que o fator lúdico atue como catalisador do processo de ensino-aprendizagem.

1.2 Fundamentos de Game Design

Os fundamentos de *Game Design* compreendem a estruturação sistemática das regras, mecânicas, fluxos de desafio e elementos estéticos que governam a experiência interativa do jogador. No contexto de jogos educacionais baseados em resolução de quebra-cabeças e narrativas históricas, o design deve equilibrar o rigor conceitual com a diversão (*fun factor*), garantindo que o desafio intelectual estimule o raciocínio sem gerar frustração excessiva.

- **Curva de Dificuldade e Fluxo (*Flow*):** Segundo a teoria do fluxo de Mihaly Csikszentmihalyi, o engajamento ideal ocorre quando há um equilíbrio entre o nível de desafio imposto pelo jogo e as habilidades do jogador. Em *Turing 's Code*, essa premissa é aplicada na progressão dos quebra-cabeças, que introduzem conceitos lógicos e de programação de forma gradual (do concreto para o abstrato).

- **Mecânicas e *Feedback* Imediato:** O projeto prioriza mecânicas intuitivas amparadas por *feedback* visual e sonoro instantâneo. Sempre que o jogador manipula uma engrenagem, um cabo ou uma linha de código lógica, o sistema reage de maneira clara, permitindo a compreensão por tentativa e erro — um pilar essencial para o aprendizado ativo e a depuração (*debugging*) mental.

- **Narrativa Ambiental e Lúdica:** A história de Bletchley Park e os princípios da computação não funcionam apenas como pano de fundo estático, mas são integradas diretamente ao design de missões e à interface (UI/UX), utilizando elementos visuais nostálgicos e estruturados para contextualizar o aprendizado técnico.

1.3 Tecnologias e Ferramentas

O desenvolvimento de um jogo educativo de pixel-art com forte apelo interativo exige a escolha de ferramentas e ecossistemas tecnológicos que aliem eficiência de desempenho, suporte a mecânicas 2D e acessibilidade de plataforma.

- **Motor de Jogo (Engine):** O projeto utiliza o GameMaker, uma plataforma de desenvolvimento reconhecida por sua alta otimização para jogos 2D em pixel-art e pela linguagem de script flexível (*GameMaker Language - GML*). O motor permite um controle preciso sobre colisões, manipulação de sprites e criação de sistemas lógicos customizados, essenciais para estruturar os simuladores de quebra-cabeça e os fluxos de código do jogo.

- **Linguagens de Programação e Lógica:** A estruturação dos códigos e das regras de negócio do jogo é fundamentada em conceitos de lógica de programação (estruturas condicionais, laços de repetição, vetores e matrizes), espelhando os fundamentos de desenvolvimento de sistemas abordados no ensino técnico.

- **Criação Visual e Recursos Gráficos:** Para a direção de arte em pixel-art, utilizam-se softwares de edição gráfica especializados na construção de *sprites*, animações quadro a quadro e paletas de cores restritas, garantindo a coesão visual retrô inspirada na estética industrial e nos anos 1990.

CAPÍTULO 2 - CONCEPÇÃO DO JOGO

2.1 Visão Geral do Jogo

Nome do jogo: Turing´s Code

Gênero: Puzzle Adventure / Edutainment

Motor Gráfico: Puzzle Adventure 2D em Pixel Art

Linguagem de Programação utilizada: GML(GameMaker Language)

Público-alvo e Classificação Indicativa: Adolecente de 15 a jovens adultos.

2.2 História e Narrativa

2.2.1 Personagens

2.2.1.1 Alan Torres

1. **Papel / Função no Jogo:** Protagonista e mente lógica do jogo. É o personagem principal controlado pelo jogador nas mecânicas de lógica e programação.
2. **Descrição Visual & Estilo Pixel-Art:** Menino jovem com roupas casuais: Moletom cinza; Calça jeans; Tênis confortáveis; Cabelos marrons desalinhados.
3. **Personalidade & Elementos de Design:** Elétrico e maníaco por jogos. Representa a lógica divertida e a intuição, sendo o condutor do raciocínio e da resolução de problemas do jogo.

2.2.1.2 Ada (Mãe de Alan)

1. **Papel / Função no Jogo:** Co-protagonista. É quem interage diretamente com as engrenagens, cabos e rotores mecânicos nos desafios do jogo.

2. **Descrição Visual & Estilo Pixel-Art:** Mulher com aparência simpática de "tia"; Cabelo marrom; Roupas confortáveis; Expressão preocupada e brava em determinados momentos de tensão.

3. **Personalidade & Elementos de Design:** Prática, engenhosa e rápida para resolver problemas físicos e mecânicos.

2.2.1.3 Théo Torres

1. **Papel / Função no Jogo:** Personagem secundário e Elo Temporal / Engenheiro Híbrido. Atua como a ponte entre o presente e 1943, traduzindo conceitos modernos de TI para a linguagem de máquinas mecânicas.
2. **Descrição Visual & Estilo Pixel-Art:** Jovem de 14 anos com visual retrô-prático; suéter sobreposto a camisa xadrez; fita adesiva e ferramentas manuais penduradas.
3. **Personalidade & Elementos de Design:** Calmo, observador e focado. Acredita que "Tudo o que é digital hoje já foi engrenagem, relé e circuito ontem". É especialista em engenharia reversa analógica e adaptação de tecnologias antigas.

2.2.1.4 Soldados e Operadores

- **Papel / Função no Jogo:** NPCs (Personagens Não-Jogáveis) responsáveis por dificultar a entrada dos jogadores nos *huts* (galpões) e áreas restritas.
- **Descrição Visual & Estilo Pixel-Art:** Figura vestindo sobretudo escuro; Chapéu de aba reta; Sempre carregando maços de papéis perfurados e relatórios criptografados debaixo do braço ou carregando cacetes se for soldado.
- **Personalidade & Elementos de Design:** Sérios, cautelosos e protetores dos segredos da instalação de Bletchley Park.

2.2.2 StoryBoard

2.3 Fases do jogo : Mecânicas e Regras de Jogabilidade

2.4 Level Design

2.5 Direção de Arte e Som

CAPÍTULO 3 - ARQUITETURA E DESENVOLVIMENTO TÉCNICO

3.1 Arquitetura do Sistema

A arquitetura de software de *Turing 's Code* foi projetada sob uma abordagem modular para garantir que o jogo tenha um bom funcionamento, jogabilidade e um código simples e funcional. O sistema é estruturado utilizando o motor **GameMaker** e a linguagem de script **GameMaker Language (GML)**, organizando-se em camadas bem definidas:

- **Camada de Lógica de Jogo (*Core Gameplay*):** Gerencia os estados globais do jogo, o progresso do jogador nos *huts*, a falha nas fases e as regras de transição entre os desafios lógicos.

- **Camada de Simuladores de Circuitos e Variáveis:** Responsável por processar a lógica por trás da manipulação de cabos, rotores e linhas de código. Utiliza estruturas de dados baseadas em vetores e matrizes para validar conexões elétricas simuladas e checar a corretude dos algoritmos construídos pelo jogador.

- **Camada de Interface e Experiência do Usuário (UI/UX):** Controla a HUD monocromática em tons verde fósforo e âmbar, a renderização de fontes monoespaçadas que simulam impressoras de matriz de pontos, e os diagramas interativos em estilo "caixa" que auxiliam na compreensão lógica.

- **Camada de Renderização Gráfica (Pixel-Art):** Gerencia a exibição de *sprites* em 2D, animações quadro a quadro dos personagens (Alan e Theo) e a ambientação retrô industrial inspirada em Bletchley Park.

3.2 Interação Interdisciplinar

O projeto de desenvolvimento de *Turing' s Code* atua como um eixo integrador do curso técnico em Desenvolvimento de Sistemas, articulando a teoria acadêmica com a prática prática de desenvolvimento por meio de quatro pilares fundamentais:

- **Lógica de Programação:** A concepção dos desafios lógicos exige o emprego prático de conceitos fundamentais da programação, como estruturas condicionais, laços de repetição, vetores, matrizes e manipulação de variáveis. O erro é tratado como um recurso pedagógico de feedback imediato.

- **Arquitetura de Computadores e Redes:** A narrativa e a mecânica resgatam os princípios físicos da computação — explorando a transição histórica entre engrenagens mecânicas, relés, circuitos analógicos e a lógica binária moderna. A divisão de papéis entre Alan (software/lógica) e Théo (hardware/analógico) materializa essa convergência tecnológica.

- **História, Sociedade e Cidadania:** A imersão no contexto histórico de Bletchley Park e na Segunda Guerra Mundial contextualiza o impacto social da tecnologia, desmistificando a computação atual e valorizando o esforço

coletivo e científico por trás das inovações que moldaram o mundo contemporâneo.

- **Matemática:**

3.3 Implementação do protótipo

A implementação prática do protótipo de *Turing's Code* segue uma metodologia ágil e incremental, dividida em três fases principais de desenvolvimento:

1. **Fase 1: Prototipagem Estrutural:**
    - Configuração do projeto no GameMaker.
    - Implementação das sprites principais(Quarto do Alan e Huts), criação dos personagens e dos movimentos básicos dos protagonistas (Alan e Théo).
    - Desenvolvimento do sistema mecânico inicial, como seriam as fases e validando a estética visual em pixel-art.

2. **Fase 2: Integração de Mecânicas:**
    - Desenvolvimento dos simuladores lógicos do *Hut 8, Hut 6 e Bletchley Park.*
    - Implementação da interface de *feedback* visual imediato e dos diálogos narrativos entre os primos e o Alan Turing.
    - Aplicação das restrições de tempo/movimento e dos elementos de depuração.

3. **Fase 3: Polimento, Testes e Otimização:**
    - Refinamento da paleta de cores da HUD, trilha sonora e efeitos sonoros dissonantes.
    - Realização de testes de usabilidade (UX) com integrantes para ajustar a curva de dificuldade e garantir o equilíbrio do fluxo (*Flow*).
    - Otimização de desempenho de *sprites* e compilação do protótipo jogável para validação final.

CAPÍTULO 4 - TESTES E RESULTADOS

4.1 Metodologia dos testes

4.2 Análise dos resultados

4.3 Ajustes e melhorias Realizadas

CAPÍTULO 5 - CONSIDERAÇÕES FINAIS

5.1 Conclusões

5.2 Dificuldades Encontradas

5.3 Possíveis alterações

REFERÊNCIAS BIBLIOGRÁFICAS

Aprendizagem baseada em jogos digitais - Marc Prensky - Editora Senac São Paulo, 10 de nov. de 2021 - 576 páginas